

LeetCode編碼：

112

系所：北聯(北醫) 生物醫學工程學系

學號：B812111001

姓名：張博翔

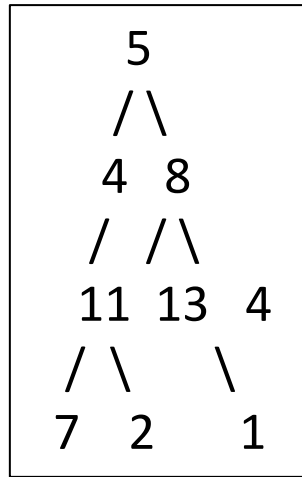


題目 (含解釋&說明)



給定一棵Binary Tree 與目標值 `targetSum`，判斷是否存在一條從Root到Leaf的路徑可使所有節點的值加總等於 `targetSum`。

- 以目標值 `targetSum = 22` 為例：



- 從根到葉共有四條路徑： $5 \rightarrow 4 \rightarrow 11 \rightarrow 7 = 27$ 、 $5 \rightarrow 4 \rightarrow 11 \rightarrow 2 = 22$ 、 $5 \rightarrow 8 \rightarrow 13 = 26$ 、 $5 \rightarrow 8 \rightarrow 4 \rightarrow 1 = 18$ 。其中 $5 \rightarrow 4 \rightarrow 11 \rightarrow 2$ 的總和恰好為 22，回傳 True。
- 需要注意當root 為 none 時，不存在任何路徑直接回傳 False，判斷到達葉節點時，必須確認子節點皆為 none，只有單側為 none 的節點不算葉節點，若在此計算判斷會錯誤。節點值可為負數，因此不能在路徑總和超過 `targetSum` 時提早剪枝，必須走到葉節點才能下結論。



- 解法一： **Recursive DFS**
- 將判斷整棵樹是否有符合條件的路徑這個問題，拆解成判斷左子樹或右子樹是否有符合條件的路徑的小問題，層層遞迴直到葉節點，屬於典型的 **Recursive DFS**。
- 每層遞迴先將當前節點的值從 **targetSum** 扣掉，得到剩餘 **targetSum**，再把這個剩餘值傳給左右子節點繼續判斷。

好處是不需要另外維護到當前節點的 **Sum**，只需要追蹤還差多少，到葉節點時若剩餘值恰好等於葉節點的值代表找到符合條件的路徑。

優點是程式碼簡潔，遞迴的結構與樹的結構對應。缺點是遞迴深度等於樹的高度 **h**，在最差情況下（不平衡，退化成線性） $h = n$ ，**call stack** 深度為 $O(n)$ ，有爆 **stack** 的風險。



- 解法二：迭代 **DFS**
- 將遞迴改寫成迭代，用明確的 **stack** 取代系統的 **call stack**，消除遞迴爆 **stack** 的風險，同時保留 **DFS** 的遍歷順序。
- 遞迴的 **call stack** 由系統管理，深度有上限且不易控制；而迭代的 **stack** 存於 **heap** 記憶體中，理論上只受系統記憶體限制，在極深的樹上更為穩健。

stack 中存放的是 **(node, Sum)**，**pop** 出一個節點後，檢查它是否為葉節點，若是則判斷累積和是否等於 **targetSum**；若不是葉節點，則將左右子節點連同更新後的 **Sum** **push** 進 **stack**。

選擇存 **Sum** 而非 **targetSum** 是因為在迭代中節點的狀態需要獨立存放在 **stack entry** 裡，累積和的語意比剩餘值更直觀。

時間與空間複雜度與遞迴相同（**O(n)** 時間，**O(h)** 空間），但消除了爆 **stack** 的問題。



- 核心是tree的遍歷，路徑只能從根走到葉不能中途停下。
遞迴讓樹的結構直接映射到 call stack，寫起直觀；
迭代則把消除了爆 stack 的問題。
- 遞迴轉迭代的思維，在 LC 94 、LC 144 樹的遍歷題也可參考。

方法	Time	Space
遞迴 DFS	$O(n)$	$O(h)$
迭代 DFS	$O(n)$	$O(h)$



```

1 class Solution(object):
2     # 解法一：遞迴 DFS
3     def hasPathSum_recursive(self, root, targetSum):
4         if not root:
5             return False
6         # 只有左右子節點皆為 None 才是葉節點
7         # 單側為 None 的節點不算葉節點，不能在此判斷
8         if not root.left and not root.right:
9             return targetSum == root.val
10        # 將當前節點的值從目標中扣除，傳給子節點繼續判斷
11        # 這樣不需要另外維護累積和，只需追蹤還差多少
12        remaining = targetSum - root.val
13        return (self.hasPathSum_recursive(root.left, remaining) or
14                self.hasPathSum_recursive(root.right, remaining))
15

```

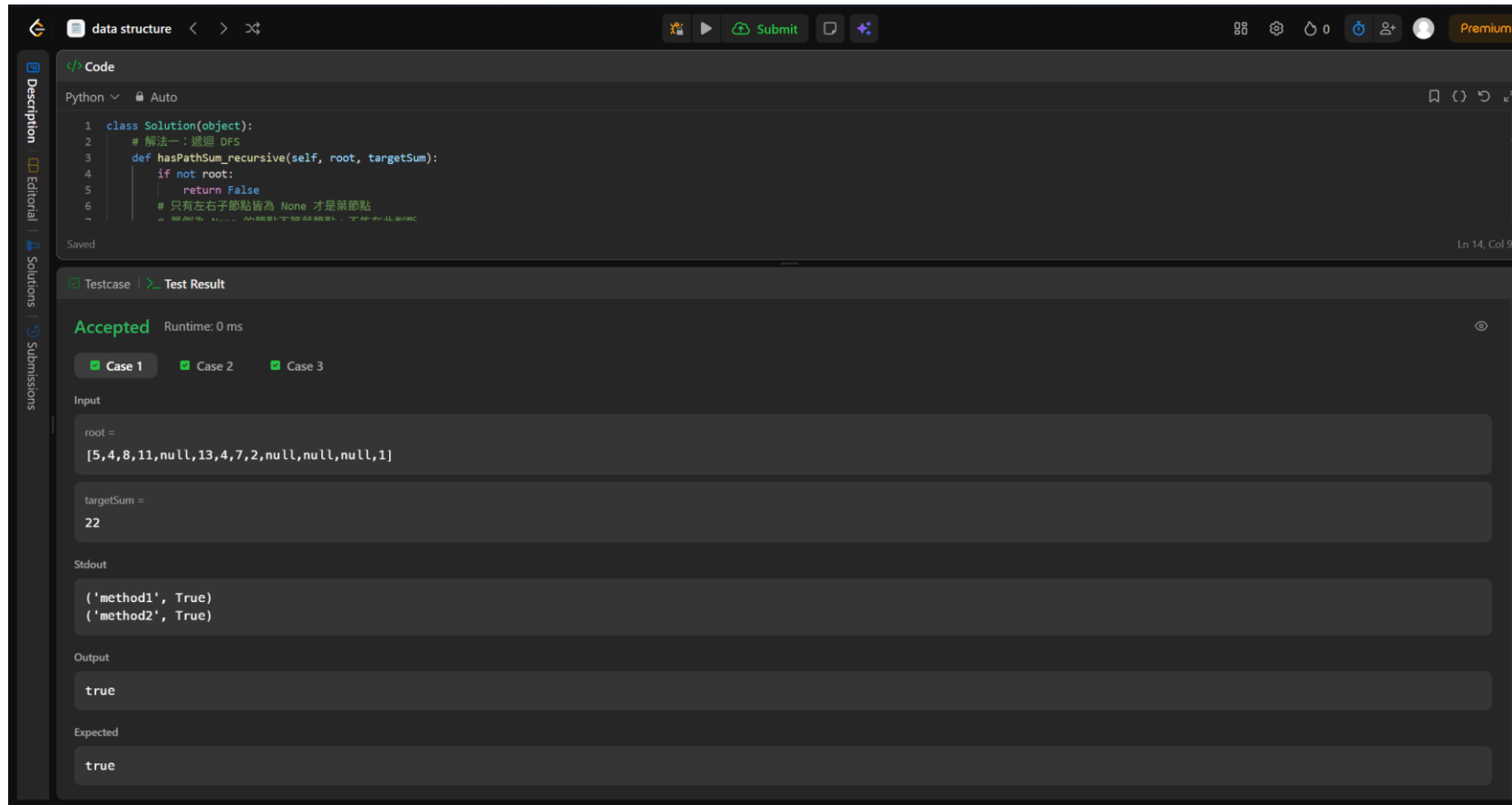
```

16 # 解法二：迭代 DFS
17 def hasPathSum_iterative(self, root, targetSum):
18     if not root:
19         return False
20     # stack 存 (節點, 從根到此的累積和)
21     # 用累積和而非剩餘值，是因為每個節點的狀態需要獨立存放
22     # 累積和的語意在多個節點同時在 stack 中時不易混淆
23     stack = [(root, root.val)]
24     while stack:
25         node, curr_sum = stack.pop()
26         # 只有到達葉節點才判斷，中途節點不能提早下結論
27         if not node.left and not node.right:
28             if curr_sum == targetSum:
29                 return True
30             continue
31         # 將子節點連同更新後的累積和一起推入 stack
32         if node.right:
33             stack.append((node.right, curr_sum + node.right.val))
34         if node.left:
35             stack.append((node.left, curr_sum + node.left.val))
36     return False
37
38 def hasPathSum(self, root, targetSum):
39     res1 = self.hasPathSum_recursive(root, targetSum)
40     res2 = self.hasPathSum_iterative(root, targetSum)
41
42     print("method1", res1)
43     print("method2", res2)
44
45     return res1

```



- 結果證明：



data structure < > Submit

Python Auto

```

1 class Solution(object):
2     # 解法一：遞迴 DFS
3     def hasPathSum_recursive(self, root, targetSum):
4         if not root:
5             return False
6         # 只有左右子節點皆為 None 才是葉節點

```

Saved Ln 14, Col 9

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

root =

```
[5,4,8,11,null,13,4,7,2,null,null,null,1]
```

targetSum =

22

Stdout

```
('method1', True)
('method2', True)
```

Output

true

Expected

true